STEAM ICAC 2026 Analysis – Iris-Lite

Abhiram Vadali, Shreyash Thakur, Subeg Singh Gill, Atharv Rajesh

Computer Science - Team C6

Date Of Submission: May 1st

Abstract

Traditional home surveillance relies on continuous recording and remote storage, an architecture that sacrifices privacy while inefficiently using embedded systems (Paolanti & Frontoni, 2020). Entry-level systems often lack audio recording, reducing event context (Mustafa, 2024). Iris-Lite instead uses a tiered pipeline on a Raspberry Pi 4B, combining OpenCV zone monitoring, a CNN audio classifier, and two-tier motion filtration to extract events from a 5-minute SHM buffer, compressed via P.E.L.I.C.A.N., a hardware-accelerated H.264 engine scaling quality by frame importance and system load. On-device processing, user-defined privacy zones, and carbon-aware charging ensure data protection and energy efficiency. A 10-minute field test confirmed stable operation across six core components, with 94.28% median CPU usage and 1.5-1.8 GB RAM, within the Raspberry Pi 4B's limits.

Sustainability

The prototype prioritizes sustainability via a threshold-based system, activating advanced analysis only when said thresholds are passed. Light algorithms capture moments from a sliding 5-minute window of 45% quality JPEGs in shared memory (SHM), maximizing longevity since RAM resists wear from read/write cycles better than traditional storage (Anderson, 2025). An SHM provides *O(1)* access with lower latency than non-volatile storage, bypassing SSD controller overhead to maintain performance under load (*Numbers Every Programmer Should Know By Year*, n.d.). The system uses the Raspberry Pi 4B's H.264 hardware encoder for compression (Halfacree, 2022), triggering deep-learning sound models and background subtraction only during high-risk events to reduce load (OpenCV, n.d.). For hardware, an ESP32-C3 (Vadali, 2026) manages carbon-aware charging, instructing the Iris-Lite to use battery power unless renewable energy is available for charging. According to the Green Software Foundation, carbon-aware technologies "do more when leveraging greener energy". This approach ensures Iris-Lite outperforms inefficient budget models, while also curbing the carbon footprint & power draw (Halfacree, 2022).

System	Storage	Compression	Power Draw	CPU (%)	Heavy	Event/Incident Context
Iris-Lite	Rolling buffer	Integrated H.264 chips, event-driven.	Less power draw, selective analysis	94.28% median usage.	Context-aware processing + Compression	Saves all clips to flash media (USB) Analyses & records audio for context
CCTV	24/7 writes, NAND cells wear out	H.265 at best, constantly runs	High power draw due to being always on, no context.	95% usage.	No processing, lots of storage used.	Earlier models used in older homes or retirement centers do not record audio.

Field Experiment

» *Testing Overview & Environment*

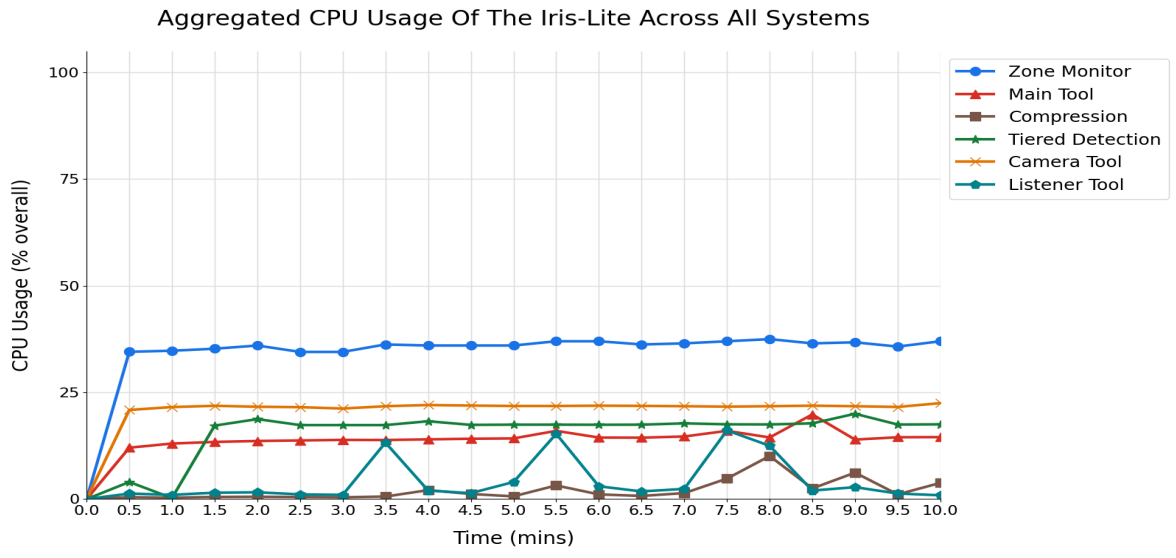
The field test was conducted over a 10-minute continuous run on a Raspberry Pi 4B with 2GB RAM, all six processes were active simultaneously. Events such as grass overlap, concerning sounds, and light changes were triggered periodically to simulate a real-world environment. In a front-porch setting, which is the most common for home surveillance (Li, 2025), a USB camera captured 1080p JPEGs and audio. GPIO pin 17 monitored the door sensor (reed switch), and pin 27 monitored the motion sensor. (*Raspberry Pi GPIO Pinout*, n.d.). CPU usage was sampled at 30-second intervals over ten minutes, while RAM usage stayed between 1.5 and 1.8 GB, meeting expectations for frame storage inside SHM.

» *Findings*

The system operated stably for 10 minutes without crashes, SHM errors, or dropped frames. An automated shell script logged CPU usage, with a median of 94.28% across the entire system (all cores). This confirms Iris-Lite's architecture fits the Raspberry Pi 4B capabilities. Package temperatures stayed at 68°C via passive and active cooling, though potential improvements were identified.

The Zone Monitor was the primary consumer at ~34-38% CPU usage, driven by heavy GSOC background subtraction. Tiered Detection utilized ~17% during tier-one, spiking to 20% for tier-two entropy analysis. The Camera Tool remained stable below 23% while querying hardware and writing to SHM blocks, while the Main Tool generally stayed under 15%, spiking to 20% due to JPEG compression backlogs. Removing the CPU-heavy OpenCV *cv2.imshow* call could reduce the total footprint to ~85% CPU.

At minute 8, the P.E.L.I.C.A.N. system peaked at 10.1% CPU usage during complex processing. The software optimized the clip using human saliency standards, while SystemGovernor stabilized the encoding via CRF adjustments for a streamlined completion.



Automatic Identification of Useful Footage

Traditional security cameras nowadays record constantly, wasting storage and footage. The Iris-Lite prototype is equipped with a system that filters important moments from a running 5-minute buffer, leaving the user with a directory of compressed clips that consume far less storage than traditional, constantly-recording CCTV systems. Systems such as a zone monitor, sound analyser, and a two-tiered footage-filtration system power this process. Sensors are incorporated into the prototype by using the on-board GPIO pins, enhancing the saliency algorithm that the P.E.L.I.C.A.N. compression software uses. The sensors used are a reed switch module (door sensor) and a basic motion sensor.

The Zone Monitor component serves as a detection layer, using OpenCV's GSOC algorithm to isolate foreground activity from the background, ignoring shadows by default (Kulkarni et al., 2024). Centroid-based distance tracking and area-sorting are used to identify and isolate primary zones (grass chosen for the experiment) within view using HSV color segmentation & kernel-based image processing. User-defined privacy zones will not be processed and will only reveal an abstract image when movement is detected. Detection is only triggered when the bottom twenty pixels of a foreground object intersect the zone mask for greater than one second. Once the threshold is met, the system generates a metadata object, containing the trigger (zone detector in this case), start time, and end time of the clip, with 5 seconds of footage being added to each end, ensuring full context.

Beyond zone monitoring, the Iris-Lite prototype uses sound processing, which is far ahead of the industry standard. A LiteRT interpreter is used to analyze live mel-spectrograms of audio beyond a volume threshold (-21.9 dBFS, approx. 72 dB SPL) using a convolutional neural network trained for 30 epochs. It is trained to identify sounds across six event categories: car screeching, screaming, gunshots, glass breaking, aggressive knocking, and dog barking, as well as an ambient background noise class, which is deemed not a threat. After analysis, if a sound matches a concern category with over 45% confidence, the software returns a CaptureClass object that contains start time, end time, and a classification label. When idle (sound is under the threshold), this component uses a minimal amount of system resources to process 44.1 kHz audio, only reaching a peak of 16.1% CPU usage during sound processing.

Finally, the proposed detection software employs a pipeline consisting of two filtration tiers to optimize the Raspberry Pi 4B's limited system resources. Tier 1 is an always-on system, which uses CNT background subtraction to identify foreground changes (with a sensitivity factor of 0.15), and EMA luminance filtering (with a running alpha factor of 0.05) to detect spikes in brightness exceeding a 90-unit threshold. If an issue is detected, Tier 2 is activated, utilizing motion entropy analysis to differentiate natural from erratic motion. Also, spatial-temporal persistence is used to verify object validity, with a requirement of a 50-frame consistent path to prevent false positives. This tiered approach allows the processor to prioritize the zone monitor and sound analyser while providing an additional layer of detection to accommodate more situations.

Video Compression Software

The Iris-Lite system incorporates a dedicated compression subsystem, P.E.L.I.C.A.N. (***P****erceptual* ***E****dge* ***L****ightweight* ***I****ntelligent* ***C****ompression and* ***A****daptive* ***N****etwork*), designed for Raspberry Pi 4B edge hardware, where continuous 1080p encoding is constrained by CPU load, thermal conditions, and memory bandwidth limitations. Rather than treating all frames uniformly, P.E.L.I.C.A.N. operates as an event-driven, perceptually adaptive pipeline that selectively encodes frames based on informational relevance, reducing storage overhead while preserving event-level fidelity aligned with principles of human visual perception (Wang et al., 2004).

Incoming video is streamed through an SHM buffer and processed using a dual-resolution architecture. Full-resolution 1080p frames are retained for encoding, while a downscaled 320×180 representation is used for lightweight analysis. Each frame is assigned a normalized importance score in [0,1], derived from motion intensity through frame differencing (Kamperman, 2010), spatial complexity (edge density), and temporal proximity to event peaks. Spatial complexity is estimated using edge-based feature extraction (Maini & Aggarwal, n.d.), while stability under rapid scene changes is maintained using exponential moving average filtering ($\alpha = 0.08$) (Hyndman & Athanasopoulos, 2021).

A System Governor monitors system load and thermal conditions to produce a unified pressure signal. Under increased pressure, P.E.L.I.C.A.N. increases compression aggressiveness, reduces encoding quality, and skips low-importance frames to maintain real-time responsiveness, consistent with edge resource management principles (Shi et al., 2016).

Video is segmented into clustered event windows using the EventCluster module, which merges temporally adjacent detections into clips, reducing fragmentation and redundant encoding. Within each window, frames are encoded using hardware-accelerated H.264 (h264_v4l2m2m) (Gupta, 2023), optimized for compression efficiency on constrained systems (Wiegand et al., 2003), with direct memory streaming to eliminate intermediate storage.

GPIO-connected motion and door sensors provide contextual signals that bias importance scoring within short windows around physical triggers (±5 seconds), prioritizing relevant activity in line with sensor-fusion-based edge systems (Gubbi et al., 2013).

Compression quality is governed by a CRF-based mapping function that translates perceptual importance and system pressure into quantization strength, enabling adaptive bitrate allocation where high-saliency frames retain detail while low-importance segments are heavily compressed or discarded. Overall, the system demonstrates real-time edge operation on Raspberry Pi 4B hardware (Raspberry Pi Foundation, n.d.), with compression dynamically adapting to scene complexity and system load while maintaining stable performance.

Edge Processing, Scalability, & Privacy

The Iris-Lite utilizes edge-processing to analyze all video and audio on-device, bypassing cloud-based algorithms. Constant processes like the CNT & GSOC background subtraction and sound monitoring keep resource usage balanced, while complex algorithms trigger only when specific thresholds are crossed. The prototype features a fully cloud-optional design without facial recognition, ensuring feeds are not at risk of unauthorized access. User-defined privacy zones allow the masking of sensitive regions, where motion is rendered as abstract movement to protect privacy during investigations. (*Configure Privacy Zones for Video Devices - Knowledge Base*, 2026) The system is easily scalable using a Raspberry Pi as a central hub (IBM, 2023). Currently, the device remains at around 94.28% total CPU usage (out of a 100% maximum across all cores) and a temperature of 68°C with all systems running. The integration of a hardware-accelerator, such as the Coral Edge TPU, would accelerate TensorFlow Lite models (Coral AI, 2020). This would allow the system to control multiple cameras at once, as stress on the Pi 4B's CPU is alleviated.

Prototype Limitations

While Iris-Lite is effective, there are some limitations in the current prototype. The system relies on basic motion detection and region-based analysis, which means it may not always correctly identify all threats. For example, it may sometimes miss important events or trigger false alerts due to algorithmic limitations, sudden lighting changes, or fast movement. The audio processing is also limited, as it is trained on a smaller set of sound data, which adversely affects its accuracy. Performance may also vary depending on the hardware used, especially on older edge devices. For this experiment, a Raspberry Pi 4B was chosen, as it provided an appropriate middle-ground between processing power and testability. Finally, while the compression system reduces storage usage, it may lower video quality in dormant areas/areas with less activity. Future improvements, such as privacy-conscious object recognition, automatically defined zones (not just grass), and an expanded set of training data, or an increasingly advanced compression software, could help make the system more accurate and reliable.

Conclusion

In summary, Iris-Lite demonstrates that home surveillance can be both effective and environmentally responsible. Through efficient code and selective activation, it reduces energy waste and storage while maintaining reliable monitoring. Rather than operating continuously, the system responds only to meaningful activity. Core components such as the analysis loop, motion classification, and audio filtering deliver intelligent responses by identifying important events. By combining low-power hardware with optimized, modular software, the prototype reduces power & storage requirements. Future additions like foreground tracking, infrared integration, or a hardware acceleration module could further strengthen its efficiency and accuracy.

References

Anderson, D. (2025, September 10). *The Battle of SSD and RAM: A Comprehensive Guide*. Autonomous. Retrieved March 26, 2026, from https://www.autonomous.ai/ourblog/the-battle-of-ssd-and-ram

Configure Privacy Zones for video devices - Knowledge Base. (2026, January 29). Knowledge Base. Retrieved March 25, 2026, from https://answers.alarm.com/Partner/Installation_and_Troubleshooting/Video_Devices/General_Video_Information/Configure_Privacy_Zones_for_video_devices

Coral AI. (2020). *Edge TPU performance benchmarks*. coral.ai. Retrieved March 26, 2026, from https://www.coral.ai/docs/edgetpu/benchmarks/

Green Software Foundation. (2025). *Carbon Aware SDK*. carbon-aware-sdk.greensoftware.foundation. Retrieved March 23, 2026, from https://carbon-aware-sdk.greensoftware.foundation/docs/overview

Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013, September). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, *29*(7), 1645–1660. ScienceDirect. 10.1016/j.future.2013.01.010

Gupta, A. (2023, October 15). *What is the difference between H.264 vs. H.265?* api.video. Retrieved April 20, 2026, from https://api.video/blog/video-trends/H264-vs-H265/

Halfacree, G. (2022). *Chris Griffith Puts the Raspberry Pi's Hardware H.264 Encoder Through Its Paces*. Hackster.io. Retrieved April 02, 2026, from https://www.hackster.io/news/chris-griffith-puts-the-raspberry-pi-s-hardware-h-264-encoder-through-its-paces-1d804e538d9e

Hyndman, R. J., & Athanasopoulos, G. (OTexts). *Forecasting: Principles and Practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

IBM. (2023, June 20). *What Is Edge Computing?* IBM.com/ca-en. Retrieved April 1, 2026, from https://www.ibm.com/topics/edge-computing

Kamperman, K. (2010, April 24). *Computer Vision: Frame differencing*. kasperkamperman.com. Retrieved April 22, 2026, from https://www.kasperkamperman.com/blog/computer-vision/computervision-framedifferencing/

Kulkarni, V., Pol, T., & Mapari, S. (2024, April). *Comparative Study of Background Subtraction Algorithm for Moving Object Detection using OpenCV*. Harbin Engineering Journal. Retrieved March 29, 2026, from https://harbinengineeringjournal.com/index.php/journal/article/view/2872

Li, A. (2025, December 29). *Where to Place Security Cameras in Home?* Reolink. Retrieved April 29, 2026, from https://reolink.com/blog/where-to-place-home-security-cameras/

Maini, R., & Aggarwal, H. (n.d.). *Study and Comparison of Various Image Edge Detection Techniques*. Scientific Research. Retrieved April 17, 2026, from https://www.scirp.org/reference/referencespapers?referenceid=2378612#:~:text=Maini%2C%20R.

Mustafa, A. (2024, April 16). *Do Security Camera Systems Record Audio?* Canadian Security Professionals. Retrieved March 30, 2026, from https://www.cspalarms.ca/blog/security-cameras/do-security-camera-systems-record-audio/

Numbers Every Programmer Should Know By Year. (n.d.). Colin Scott. Retrieved April 1, 2026, from https://colin-scott.github.io/personal_website/research/interactive_latency.html

OpenCV. (n.d.). *Background subtraction using bgsegm.* docs.opencv.com. Retrieved 3 21, 2026, from https://docs.opencv.org/3.4/d1/dc5/tutorial_background_subtraction.html

Paolanti, M., & Frontoni, E. (2020). *Video Surveillance*. ScienceDirect. Retrieved April 12, 2026, from https://www.sciencedirect.com/topics/computer-science/video-surveillance

Raspberry Pi Foundation. (n.d.). *Camera*. Raspberry Pi. Retrieved April 5, 2026, from https://www.raspberrypi.com/documentation/accessories/camera.html

Raspberry Pi GPIO Pinout. (n.d.). Raspberry Pi GPIO Pinout. Retrieved April 1, 2026, from https://pinout.xyz/

Shi, W., Cao, J., Zhang, Q., Li, V., & Xu, L. (2016, October). Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, *3*(5), 637–646. IEEE Xplore. 10.1109/JIOT.2016.2579198

Vadali, A. S. (2026, 04 30). *CarbonAwareCharge*. GitHub.com. https://github.com/AbhiramV010/CarbonAwareCharge

Vadali, A. S., & Thakur, S. (2026, April 30). *2026-STEAM-IC*. GitHub. Retrieved April 30, 2026, from https://github.com/AbhiramV010/2026-STEAM-IC

Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004, April). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, *13*(4), 600-612. IEEE Xplore. 10.1109/TIP.2003.819861

Wiegand, T., Sullivan, G. J., Bjøntegaard, G., & Luthra, A. (2003, July). Overview of the H.264/AVC video coding standard. *IEEE Transactions on Circuits and Systems for Video Technology*, *13*(7), 560-576. IEEE Xplore. 10.1109/TCSVT.2003.815165